MetroInsights Phase 6 - System Design, Performance and Scalability

Project-specific architecture guide | Repository revision 4405d167fc323f33be76e2c7f6614fd21cc574a8

Purpose and scope

This guide explains how MetroInsights behaves today and how it could evolve if actual usage or measured response times justify the change. It does not claim that a cache, database, queue, cloud object store, monitoring service, or autoscaling setup already exists.

The current product is a single Flask web application. It loads fitted model files and small station/operations datasets when its Python modules import, computes a station forecast and timetable recommendation on request, calculates shortest-path estimates, and returns HTML or JSON. The large generated training CSV is used by offline model training; the web request path loads the committed fitted artifacts and compact station metadata instead. This distinction matters for deployment memory, startup, and storage planning.

1. Current system at a glance

Request path

1. Browser sends station, date, time, and line-focus values to `/api/predict` (`app/static/js/dashboard.js`, `app/routes.py`). 2. Flask validates/parses the date and time and obtains calendar/service-window context. 3. `app/forecasting.py` builds feature values and asks the already-loaded scikit-learn model for hourly predictions. A full-day profile evaluates 19 operating-hour labels (05:00 through 23:00). 4. `app/scheduling_optimizer.py` allocates the estimated demand across lines and calculates service recommendations from configured capacities and headways. 5. `app/travel_graph.py` has built a line-expanded graph at import/startup; Dijkstra computes route-time estimates from the selected station. 6. Flask serializes the response as JSON; browser code updates the dashboard.

What is loaded and when

• Model (`models/footfall_forecaster.joblib`), metadata, station table, evaluation metrics, operations table, and route graph are loaded or built during module import.

• The 130 MB generated CSVs in `data/synthetic/` and `csvFiles/` support pipeline/training work. The request code does not read those files on each prediction.

• The `Procfile` starts `gunicorn run:app`. Deployment memory and cold-start behavior still need measurement on the actual Render plan.

• Gunicorn worker processes normally have separate Python heaps. Do not assume that the loaded model and Pandas objects are shared between workers.

Current guarantees and non-guarantees

• The model is trained on synthetic simulation data; inference latency does not make its predictions operationally validated.

• Travel durations are shortest-path estimates over project topology, average-speed assumptions, and interchange penalties; they are not live timetable journey times.

• No persistent database, distributed cache, task queue, autoscaler, or application performance monitoring is present in the core request flow.

• The dashboard asks for a full response after scenario changes. Frontend stale-response protection, where used, is not the same as cancelling backend computation.

2. Establish a baseline before optimizing

Measure the user-visible path

Capture a baseline on the deployed service and a local run using the same station/date/time inputs. Record at least:

• Browser-observed request duration and whether time is spent waiting for the service to wake.

• Server duration by request route, ideally p50, p95, and p99 over a representative window.

• Cold-start time separately from warm request time.

• Response status, response byte size, and errors/timeouts.

• CPU and resident memory by process and by Gunicorn worker.

• Concurrent requests and worker saturation.

Use a small repeatable input set: ordinary station, interchange station, morning peak, evening peak, closed hour, and an invalid request. Compare warm and cold requests; do not infer throughput from a one-off browser click.

Profile code before redesign

Use a local profiler such as `cProfile` to locate Python time in a representative API call, and use `time.perf_counter()` around coarse boundaries when investigating. Profile separately:

• construction of the full-day demand profile;

• model inference and feature-frame construction;

• scheduling calculations;

• Dijkstra search;

• JSON serialization.

Avoid permanent noisy timing prints in the request path. Add structured timing/log fields or use a profiler in a controlled run, then remove diagnostic instrumentation or gate it appropriately.

Example experiment plan

1. Warm the service, send a fixed set of requests one at a time, repeat enough times to see spread, and record median and p95. 2. Repeat with increasing concurrency while observing CPU, memory, queueing, and error rate. 3. Repeat after idle/sleep to capture Render cold-start behavior. 4. Change one implementation detail at a time; rerun the identical inputs and compare response content as well as time. 5. Keep a before/after note with environment, worker count, request mix, and sample count. A faster but numerically different response is not automatically a safe optimization.

3. Likely costs and practical optimization order

Cost A: repeated full-day predictions

`/api/predict` needs a selected-hour estimate and a full-day curve. The profile calls hourly forecasting repeatedly for 19 hours; each call constructs a one-row Pandas DataFrame and invokes the estimator. This is simple and small, but it can be redundant when the user changes only the selected minute or line focus.

Low-risk options after profiling: build the 19-row feature matrix once and call `model.predict()` in a batch; reuse the selected hourly value from the profile; avoid recomputing an unchanged profile when only presentation state changes. Preserve the training feature order and compare every output with the current implementation on fixed cases.

Trade-off: batch inference lowers per-call overhead but does not solve model accuracy, and careless feature refactoring can change semantics.

Cost B: Dijkstra on every request

The graph is compact relative to a city-wide production network, and the project currently has about 226 stations. Dijkstra is a reasonable choice at this size. `/api/predict` nevertheless recomputes shortest paths for the same origin each time.

Low-risk option: precompute all-origin travel-time maps once after graph construction, or memoize by origin because graph topology and weights are static for a process. Invalidate if topology, coordinates, speed, or interchange penalties change. Verify equality to the current solver.

Trade-off: an all-pairs map trades a small amount of startup work/memory for faster requests. Measure instead of assuming it matters.

Cost C: module-import work and cold start

Loading scikit-learn, Pandas, and joblib artifacts plus building the graph happens during imports. Each fresh worker repeats process initialization. This can increase cold startup and multiplies model/table memory across workers.

Options: measure first; keep heavyweight imports out of rarely used paths if possible; lazy-load only if it materially improves startup and has safe thread/process semantics; use fewer workers if memory is the limiting factor; choose a paid instance only when actual availability/latency needs justify its cost.

Cost D: payload and browser rendering

The response includes profile curves, recommendations, and up to hundreds of destination travel times. A few hundred short records are not intrinsically large, but full response and DOM rendering should be measured. The client currently renders a large travel list and supports local filtering.

Options: compress HTTP responses at the proxy/app server, reduce duplicate fields, provide a dedicated paginated/search endpoint if the list becomes expensive, and avoid rebuilding unrelated chart/table elements. Preserve accessibility and complete station search behavior.

Cost E: training-data generation, not serving

The large CSV has millions of synthetic rows and can dominate offline preprocessing/training memory and build time. It is ignored by Git and is not needed to serve predictions when the compatible model/metadata artifacts are present.

Options for retraining: profile pipeline memory/time; select only needed columns; use vectorized calculations; write/read partitions or chunks if an operation can be made streaming; keep raw/generated data in suitable external storage with versioned provenance. Do not run model training inside a web request or an ephemeral free web instance.

4. Memory and worker sizing

Current memory model

A web worker holds Python/runtime libraries, model parameters, metadata dictionaries, station/operations tables, route graph, and request-local features/response objects. Requests may briefly add Pandas frames and JSON objects. With multiple independent processes, much of the baseline is multiplied by worker count. Operating-system copy-on-write behavior can reduce physical duplication in some deployments, but it must not be treated as a guaranteed capacity plan.

Practical sizing method

1. Read steady-state RSS after startup and under a few concurrent requests. 2. Observe peak RSS during startup and request bursts. 3. Measure the memory budget of the actual Render instance and leave headroom for runtime variance. 4. Select worker count based on both CPU and memory; more workers can increase throughput only until CPU, memory, or other bottlenecks saturate. 5. Watch for worker restarts, out-of-memory events, and latency spikes.

The free Render screenshot previously showed 512 MB RAM and 0.1 CPU, but service plans can change. Treat actual dashboard/service settings as authoritative at deployment time. A process that starts successfully can still be unstable under concurrent requests.

5. Caching: what is safe and what is not

Safe candidate: route maps

Static topology and weights make shortest-time results deterministic within one application revision. Cache by canonical origin, or compute all origins once. Key the cache with a topology/version identifier if runtime topology becomes configurable.

Candidate: forecast profiles

A forecast profile depends on station, date, model version, metadata/data version, and calendar context. A cache key should include those values. Keep the cache bounded (LRU/TTL or explicit maximum entries) so arbitrary dates/stations cannot grow memory indefinitely. In a multi-worker deployment each process has its own in-memory cache; it is not shared or persistent.

Avoid: caching the entire request blindly

The selected minute, service-open state, corridor focus, and calendar context can affect parts of the response. Cache only stable subresults, then construct request-specific fields correctly. Test date boundaries, service closure, unknown station handling, and artifact upgrades.

When Redis or another shared cache makes sense

Only introduce a shared cache if measurement shows repeated expensive computation across workers or instances and the operational benefit exceeds added cost, credentials, network calls, and failure modes. Current topology size and model latency should be measured first; Redis is not a prerequisite for this project to scale to modest use.

6. Synchronous API versus background jobs

Current: synchronous prediction

A browser request does its work and returns one response. This is the right simple design while work remains short and users need an immediate result. It has fewer moving parts and easier debugging.

Keep synchronous when

• measured p95 response is comfortably within user expectations;

• prediction and route calculation are CPU-light at expected request volume;

• no long retraining or bulk import is occurring in the web process.

Add background processing only for long work

Offline training, large CSV generation, batch forecasts, and expensive report exports are natural background jobs if they become user-triggered or too slow. A job design needs a durable job store/queue, worker process, status endpoint, retries/idempotency, result storage, and monitoring. A queue alone does not make a task reliable.

For training, prefer a separate controlled pipeline or scheduled job that produces versioned artifacts, validates them, and promotes them atomically. Do not let concurrent web requests overwrite the live model files.

7. Data and model-serving evolution

Current pattern

• Offline scripts clean data, generate simulation rows, train/evaluate the model, and write joblib model/metadata artifacts.

• Web workers load those artifacts at startup and serve forecasts in-process.

• Serving does not need the 130 MB generated CSV, but reproducible retraining does need access to the corresponding source/generated data and code.

Sensible next steps

1. Record model version, feature schema, training-data version, source revision, and metrics alongside every artifact. 2. Validate artifact compatibility and required files during startup; fail with a clear deployment error rather than serving mismatched feature metadata. 3. Keep model release atomic: build and validate a candidate artifact, then deploy the code/artifact pair together. 4. Add observed DMRC or otherwise verified operational data before presenting forecast scores as real-world accuracy. 5. Keep artifacts small and process-local until inference volume or independent model lifecycle warrants a separate model service.

When a separate model-serving service wins

Consider separation if model dependencies make the web app heavy, multiple products share the model, independent release cadence is valuable, GPU or specialized inference is needed, or model traffic scales differently from dashboard traffic. It adds a network hop, service discovery, auth, compatibility/versioning, and another failure boundary. For this tabular model and current scale, in-process loading is the simpler starting design.

8. Database and durable state

The current dashboard is primarily a calculation/reporting app; no database is required for the deterministic forecast request itself. Adding a database solely to appear scalable adds schema, migrations, backups, and operational work without an evident need.

A database becomes useful when the product must save user scenarios, feedback, verified passenger observations, audit history, or model releases. Separate concerns:

• relational database for users, scenario metadata, and structured observations;

• object storage for large immutable source files and model artifacts;

• cache for derived, recomputable results;

• queue for asynchronous work.

Define retention, access control, backups, data provenance, and migrations before storing sensitive or operational data. Avoid persisting large generated CSVs as relational rows without a query use case.

9. Deployment, reliability, and observability

Current deployment shape

`Procfile` runs Gunicorn. There is no in-repository evidence of a worker queue, persistent disk, automatic horizontal scaling policy, or application monitoring service. Render free services can sleep after inactivity and have plan-specific memory/CPU constraints; verify current platform settings rather than assuming a fixed policy.

Minimum reliability improvements

• Add a lightweight health/readiness endpoint that confirms required artifacts loaded and the app can answer a minimal check.

• Log request ID, route, status, duration, and safe scenario metadata; do not log secrets or unnecessarily detailed personal data.

• Track p50/p95 latency, error/timeout rate, worker restarts, CPU, RSS, cold starts, and deployment revision.

• Set reasonable request/worker timeouts and ensure errors become useful HTTP responses.

• Keep a rollbackable release and check health after every deployment.

• Test startup with the exact production command and with required model/data files present.

Graceful degradation

If a nonessential route-time calculation fails, decide whether the forecast response can still be returned with route times marked unavailable. If core model artifacts are missing or incompatible, fail readiness and report the failure; silently returning plausible-looking fallback predictions would be misleading. If a future shared cache is unavailable, bypass it and compute locally when safe.

10. A measured scaling ladder

Level 0 - Current application

One Flask/Gunicorn service, in-process model, small static files, CSV-backed offline training pipeline, synchronous request path. Appropriate as a prototype/demo while traffic and performance remain modest.

Level 1 - Make current service efficient and diagnosable

Measure requests, batch full-day predictions, cache or precompute static route maps if profiling supports it, control worker count for RAM, set timeouts, and add health/structured request logging. This is the likely first step for modest growth.

Level 2 - Separate offline jobs and artifact storage

Run training/data generation outside request workers; store source data and versioned artifacts in durable object storage or a controlled release artifact store. Deploy web code and model artifact as a tested pair.

Level 3 - Add durable product data

Introduce a managed database only for persisted scenario/observation/history needs. Add backup, migration, access, and retention procedures.

Level 4 - Split services or scale horizontally

Add multiple web instances behind a load balancer, shared cache only for measured cross-worker benefit, and separate model or job services only when independent scaling/reliability needs justify them. Add distributed tracing and alerting as service boundaries grow.

Do not jump to Kubernetes, microservices, Kafka, or distributed model serving for a 226-station calculation app without a measured requirement. Operational complexity is itself a cost.

11. Interview-ready system-design prompt

Prompt: How would you serve MetroInsights forecasts to ten times the current user volume?

Strong answer outline: First establish current p95/cold-start latency, request rate, CPU, RSS, and error rate. The current Flask process loads a small fitted tabular model and static network data at startup; the 130 MB synthetic training CSV is not read per web request. Profile the synchronous path. Batch the 19 hourly model predictions and cache deterministic route times if they are measurable costs. Tune Gunicorn worker count against the instance memory limit because each worker can load its own model and tables. Keep model retraining outside web requests. If one instance saturates, add web instances and ensure requests are stateless; then introduce a shared cache only if repeated work across processes merits it. Add persistent storage/queue only for new requirements such as saved scenarios or user-triggered batch jobs. Verify output parity and observe p95, errors, CPU, and memory after each step.

Follow-up: What if it is slow only after inactivity? Diagnose cold start/wake-up separately from warm computation. A paid always-on plan or startup optimization may address that, while a database or queue would not.

Follow-up: What if memory is exhausted? Measure per-worker RSS and startup peak, reduce worker count or model/data footprint, and select suitable compute. Do not increase workers blindly.

Follow-up: What if the model artifact changes while requests are running? Deploy an immutable, versioned code/artifact pair and replace workers through a controlled release; avoid in-place mutation of files loaded by active workers.

12. Exercises and mastery checklist

Practical exercises

1. Draw the request flow from `dashboard.js` through `/api/predict` to forecast, scheduling, graph routing, and JSON response. 2. Time each major stage for one cold run and repeated warm runs; record p50 and p95 with environment details. 3. Compare single-worker and multi-worker RSS; explain the memory trade-off. 4. Batch the 19 profile feature rows in a temporary experiment and assert output equality against the scalar implementation for several stations/dates. 5. Precompute route results in an experiment and verify every origin/destination time matches current Dijkstra results. 6. Calculate an LRU cache key for a forecast profile and list every input/version that must invalidate it. 7. Describe deployment behavior when model files are missing, incompatible, or rolled back. 8. Propose a reliability dashboard with five actionable metrics and an alert threshold rationale.

Mastery checklist

- [ ] I can distinguish offline training costs from per-request serving costs.
- [ ] I can name the current request stages and startup-loaded objects.
- [ ] I can describe why worker count affects both throughput and memory.
- [ ] I can explain why profiling precedes caching or service splitting.
- [ ] I can propose a bounded, correctly keyed cache and its invalidation rules.
- [ ] I can choose synchronous requests versus jobs based on measured duration and user needs.
- [ ] I can explain safe model artifact versioning and rollback.
- [ ] I can scale in small steps and state the evidence needed to justify each one.
- [ ] I can distinguish implemented architecture from future design proposals.